For international companies: available via legal entities in Cyprus, Georgia, or Dubai. Deel also works.

Arkadiy Pechnikov

Senior Python Backend Engineer | High-Load Distributed Systems

Remote / Open to Relocation | arkptz@gmail.com | github.com/Arkptz | linkedin.com/in/arkptz | t.me/arkptz

---

PROFESSIONAL SUMMARY

Python backend engineer who builds systems that actually handle load. Scaled a Temporal.io platform from ~10 to 200 jobs/sec (20x), wrote a Rust computation module for a 20x speedup over baseline, and built an automated API client pipeline (mitmproxy → OpenAPI → SDK) that cut third-party integration from ~1 month to ~1 week across 20+ undocumented APIs. Comfortable with Kubernetes, observability stacks (Grafana/VictoriaMetrics/Loki), and enough protocol reverse engineering and binary analysis that picking up unfamiliar systems is usually the easy part. After running a 9-person team as technical lead, I'm focused on deep IC work inside a strong engineering org.

TECHNICAL SKILLS

Languages: Python, Rust, JavaScript/TypeScript, SQL
Frameworks & Libraries: FastAPI, aiohttp, asyncio, Celery, Temporal.io, Pydantic, SQLAlchemy
Databases: PostgreSQL, Redis, MySQL, Elasticsearch, TimescaleDB
Cloud & DevOps: Kubernetes, Docker, FluxCD, GitHub Actions, CI/CD, Helm, Cloudflare, MinIO, NixOS, k3s, Terraform
Observability: Grafana, Victoria Metrics, Loki, Sentry

PROFESSIONAL EXPERIENCE

Senior Backend Engineer (Contract)

Fenixwb - Wildberries slot booking service - 200 jobs/sec, 10K+ users | Oct 2024 - Sep 2025 | Remote

- Inherited an unstable monolith prototype pushing ~10 tasks/sec - redesigned the entire backend on Temporal.io with ~10 microservices integrated with Elasticsearch and MySQL
- Pushed throughput from 10 to 200 background jobs/sec (20x), the threshold that let the product handle real user load and start onboarding paying customers
- Disassembled a WASM captcha binary in Ghidra, then rewrote the solver in Rust - execution dropped from 2s to 0.1s (20x faster), giving clients higher booking success rates than competing bots
- Migrated from MongoDB to PostgreSQL - simpler queries, fewer ops headaches for a 3-person backend team
- Traced a critical throughput bottleneck via hypothesis-driven load testing - the root cause fix is what took us from ~10 to 200 tasks/sec
- Went from zero monitoring to production-grade observability: Grafana + Victoria Metrics + Loki with custom dashboards and alerting - incidents detected in ~3 minutes vs previously hours, MTTR dropped from days to hours

Tech: Rust, Python, Temporal.io, Elasticsearch, MySQL, PostgreSQL, asyncio, Grafana, Victoria Metrics, Loki, Docker, Kubernetes

Senior Python Developer / Technical Lead

Cryptosofters - Distributed automation platform with visual workflow builder - near-production, drag-and-drop operation pipelines | May 2023 - Nov 2024 | Remote

- Designed distributed automation platform with drag-and-drop workflow constructor - users configured chains of multi-step operations without code. Built to near-production before investment stopped
- Under the hood: event-driven state machine with per-entity operation queues, automatic retry with exponential backoff, and operation rollback on partial failures - the engine that made drag-and-drop orchestration reliable enough for production use
- Zero security incidents across the platform's lifetime. Architected per-user isolation, credential management, and operation signing in a multi-tenant environment where investor confidence hinged on it
- Reverse-engineered undocumented third-party services via traffic analysis and protocol inspection - often the only way to integrate systems with no SDK, no docs, and no public API
- Architected the automated API client pipeline: mitmproxy traffic capture → OpenAPI spec → type-safe Python SDK for 20+ undocumented third-party APIs. Cut integration from ~1 month to ~1 week (75%) - the force multiplier that made a 9-person team viable on a $100K budget
- Integrated 20+ external service APIs with a unified interface - per-integration time dropped from ~1 week to ~1 day (~80%), and any team member could add services, not just the specialists
- Introduced code review process and CI/CD pipelines for a team of 9 - regression bugs dropped noticeably and new engineers onboarded faster

Tech: Python, mitmproxy, OpenAPI, asyncio, PostgreSQL, Redis, Docker

Middle+ Python Developer / Tech Lead

Vitrinagram - Telegram MiniApp e-commerce platform - multi-tenant storefronts, 1,000+ concurrent users per store | Nov 2021 - Apr 2023 | Remote (Moscow-based)

- • Built multi-tenant Telegram MiniApp backends handling 1,000+ concurrent users per store - horizontal scaling via Celery + Kubernetes meant cost grew linearly with tenants, not exponentially
- • Automated tenant provisioning: FluxCD + Cloudflare API pipeline, ~3 minutes from commit to live subdomain
- • Developed an S3-compatible proxy API on MinIO for tenant-isolated file storage - eliminated the shared-storage security risk and cut costs vs managed S3
- • Per-tenant observability out of the box: Grafana + Victoria Metrics + Loki + Sentry dashboards clients could use for self-service debugging - ~40% fewer support tickets once clients could self-debug
- **Tech:** Python, FastAPI, Celery, PostgreSQL, Redis, Docker, Kubernetes, FluxCD, Grafana, Victoria Metrics, Loki, Sentry, MinIO, Cloudflare

Python Backend Developer

Freelance - High-throughput data processing, infrastructure automation, and backend systems | Jan 2018 - Present

- • Built distributed task orchestration system for a client with multiple investors: 2,000+ concurrent entities, 5,000+ operations/day, fully autonomous - equivalent of eliminating a full-time ops person
- • Reverse-engineered a financial data provider's WebSocket protocol and deobfuscated their JavaScript - collected 260M+ time-series data points in ~1 week, spanning 20+ years of market data. Comparable feeds charge $600+/month
- • Led 3 Go developers building a low-latency data processing system with sub-100ms response times - cross-language coordination (Python orchestration, Go hot path) where milliseconds determined outcomes
- • Operators needed consistent state across 10+ service accounts - built an automated replication system mirroring operations in real-time with near-zero drift between source and mirror
- • Developed a digital payment gateway with near-instant confirmation for a gaming platform - settlements in seconds vs traditional processing delays (delivered, pre-launch)
- • Chose Rust for the high-throughput data ingestion service because Python couldn't sustain the throughput - 300 blk/s from distributed event sources, production ingestion pipeline
- • Deployed single-node k3s on NixOS with declarative config, Terraform provisioning, and GitOps workflow - reproducible infra, entire cluster from zero in under an hour
- • Provisioned Kubernetes clusters on cloud platforms using Terraform for multiple clients - standardized the infra setup, cut provisioning time across engagements
- • Building quantitative backtesting infrastructure on nautilus-trader with 260M+ data points stored in TimescaleDB for strategy validation on financial markets
- **Tech:** Python, Rust, JS/TS, PostgreSQL, Redis, asyncio, NixOS, k3s, Terraform, TimescaleDB, Docker

PROFESSIONAL DEVELOPMENT

Machine Learning Specialization - Stanford University (via Coursera) (2026)

Reinforcement Learning for Trading Strategies - New York Institute of Finance (2026)

Using Machine Learning in Trading and Finance - New York Institute of Finance (2026)

LANGUAGES

Russian: Native | English: Upper-Intermediate (B2)